

Tutorial: Del texto al árbol — Intérprete, Compilador y Patrón Visitante

Objetivo

Este tutorial construye, paso a paso, un sistema que evalúa expresiones matemáticas como `3 + 4 * (10 - 2)`. El recorrido tiene tres etapas con un propósito pedagógico deliberado:

- 1. **Intérprete** — un analizador descendente recursivo (RDP) que evalúa la expresión directamente mientras la lee. Es el punto de partida más simple.
- 2. **Compilador** — el mismo RDP modificado para producir un árbol de sintaxis abstracta (AST) en lugar de un resultado numérico. El árbol es una representación que puede procesarse múltiples veces.
- 3. **Visitante** — una interfaz genérica que desacopla las operaciones sobre el árbol (evaluar, imprimir, optimizar...) de la estructura del árbol mismo.

A lo largo del camino, cada etapa introduce un concepto nuevo de POO: jerarquía de clases abstractas, funciones locales recursivas, e interfaces con tipo genérico.

1. El problema: sintaxis de las expresiones

Antes de escribir código, hay que definir con precisión qué expresiones vamos a entender. Eso se hace con una **gramática formal**. La gramática no es burocracia — es exactamente el esquema que le vamos a dar al algoritmo.

1.1 Gramática

```
expresion  → termino ( ('+' | '-') termino )*
termino    → unario  ( ('*' | '/') unario  )*
unario     → '-' unario | primario
primario   → NUMERO  | '(' expresion ')'
```

Cada regla define un **nivel de precedencia**:

Regla	Qué reconoce	Precedencia
expresion	<code>+</code> y <code>-</code>	más baja
termino	<code>*</code> y <code>/</code>	media
unario	<code>-</code> unario	alta
primario	números y <code>(...)</code>	más alta

El hecho de que `termino` aparezca *dentro* de `expresion` —y no al revés— es lo que hace que la multiplicación tenga mayor precedencia que la suma. No es una regla especial del algoritmo: es una consecuencia directa de la estructura de la gramática.

1.2 Cómo la gramática sugiere el algoritmo

Cada regla de la gramática se convierte en una función. La recursión de la gramática se convierte en recursión de las funciones. El árbol de llamadas en tiempo de ejecución es el árbol de la expresión.

```
expresion("3 + 4 * 2")
├─ termino("3")           → 3
├─ '+' encontrado
├─ termino("4 * 2")
│   ├─ unario("4")        → 4
│   ├─ '*' encontrado
│   └─ unario("2")        → 2
│       → 4 * 2 = 8
└─ → 3 + 8 = 11
```

Esta correspondencia directa entre gramática y código es la esencia del **análisis descendente recursivo**.

1.3 Decisiones de diseño

Antes de codificar cada paso, vale la pena explicitar las decisiones que tomamos:

No hay tokenizador separado. El analizador lee la cadena de texto directamente, carácter a carácter. En lenguajes reales, separar la tokenización del análisis simplifica el código. Aquí lo integramos para que el foco esté en el algoritmo recursivo.

Funciones locales para las utilidades. `SaltarEspacios`, `Detectar` y `Consumir` son funciones locales dentro del método principal. No son métodos de la clase porque no necesitan ser públicas ni reutilizadas desde afuera; son detalles de implementación del algoritmo. Las funciones locales en C# capturan variables del método que las contiene — en este caso, el texto y la posición actual — sin necesidad de pasarlos como parámetros en cada llamada.

Primary constructors para el AST. Los nodos del árbol son objetos de datos: reciben sus hijos en el constructor y los exponen como propiedades. El primary constructor de C# 12 hace eso en una sola línea.

El patrón Visitante con interfaz genérica. En lugar de agregar métodos `Evaluar` y `Imprimir` directamente a los nodos, usamos una interfaz `IVisitante<T>`. Esto permite agregar nuevas operaciones sobre el árbol sin modificar las clases del AST.

2. Paso 1 — El intérprete: evaluación directa

El intérprete es el RDP en su forma más pura: lee la expresión de izquierda a derecha y produce un resultado numérico sin construir ninguna estructura intermedia.

2.1 Las funciones de apoyo

Antes de las funciones recursivas necesitamos tres utilidades que manejan el cursor sobre el texto:

```
csharp

void SaltarEspacios() {
    while (pos < texto.Length && texto[pos] == ' ')
        pos++;
}

// Detectar: avanza si el próximo carácter no espacial es 'c', sino no toca nada
bool Detectar(char c) {
    SaltarEspacios();
    if (pos < texto.Length && texto[pos] == c) {
        pos++;
        return true;
    }
    return false;
}

// Consumir: igual que Detectar pero lanza excepción si el carácter no aparece
char Consumir(char esperado) {
    SaltarEspacios();
    if (pos >= texto.Length)
        throw new FormatException(
            $"Se esperaba '{esperado}' pero se llegó al final de la expresión.");
    if (texto[pos] != esperado)
        throw new FormatException(
            $"Se esperaba '{esperado}' pero se encontró '{texto[pos]}' en posición {
    return texto[pos++];
}
```

La diferencia entre `Detectar` y `Consumir` es importante. `Detectar` se usa cuando la presencia del carácter es *opcional* — decide qué rama tomar en el análisis. `Consumir` se usa cuando el carácter *debe* estar — como el `)` que cierra un paréntesis.

2.2 Las funciones recursivas

Cada regla de la gramática se convierte en una función local que retorna `double`:

```
csharp
```

```

double Expresion() {
    double resultado = Termino();    // obligatorio: al menos un término

    while (true) {
        if (Detectar('+')) resultado += Termino();
        else if (Detectar('-')) resultado -= Termino();
        else break;
    }
    return resultado;
}

double Termino() {
    double resultado = Unario();

    while (true) {
        if (Detectar('*')) resultado *= Unario();
        else if (Detectar('/')) {
            double divisor = Unario();
            if (divisor == 0) throw new DivideByZeroException("División por cero.");
            resultado /= divisor;
        }
        else break;
    }
    return resultado;
}

double Unario() {
    if (Detectar('-')) return -Unario();    // recursión para '-' repetido: --3 = 3
    return Primario();
}

double Primario() {
    SaltarEspacios();

    if (Detectar('(')) {
        double valor = Expresion();    // recursión: subexpresión entre paréntesis
        Consumir('(');
        return valor;
    }

    // Leer un número: dígitos con punto decimal opcional
    int inicio = pos;
    while (pos < texto.Length && (char.IsDigit(texto[pos]) || texto[pos] == '.'))
        pos++;
    if (pos == inicio)
        throw new FormatException(

```

```
        $"Se esperaba un número en posición {pos}, se encontró '{texto[pos]}'."
    }
    return double.Parse(texto[inicio..pos], System.Globalization.CultureInfo.InvariantCulture)
}
```

El bucle `while (true) { if Detectar... else break; }` en `Expresion` y `Termino` implementa la asociatividad izquierda: `3 - 2 - 1` se evalúa como `(3 - 2) - 1 = 0`, no como `3 - (2 - 1) = 2`.

La recursión en `Unario` implementa la asociatividad derecha del menos unario: `--3` se evalúa como `-(-(3)) = 3`.

2.3 Implementación completa

csharp

```

public class Interprete {
    private string texto = "";
    private int pos = 0;

    public double Evaluar(string expresion) {
        this.texto = expresion;
        this.pos = 0;

        double resultado = Expression();

        SaltarEspacios();
        if (this.pos < this.texto.Length)
            throw new FormatException(
                $"Carácter inesperado '{this.texto[this.pos]}' en posición {this.pos}");

        return resultado;

        // — Utilidades —————

        void SaltarEspacios() {
            while (this.pos < this.texto.Length && this.texto[this.pos] == ' ')
                this.pos++;
        }

        bool Detectar(char c) {
            SaltarEspacios();
            if (this.pos < this.texto.Length && this.texto[this.pos] == c) {
                this.pos++;
                return true;
            }
            return false;
        }

        char Consumir(char esperado) {
            SaltarEspacios();
            if (this.pos >= this.texto.Length)
                throw new FormatException(
                    $"Se esperaba '{esperado}' pero se llegó al final.");
            if (this.texto[this.pos] != esperado)
                throw new FormatException(
                    $"Se esperaba '{esperado}' pero se encontró " +
                    $"{this.texto[this.pos]} en posición {this.pos}.");
            return this.texto[this.pos++];
        }

        // — Gramática —————
    }
}

```

```

double Expresion() {
    double r = Termino();
    while (true) {
        if (Detectar('+')) r += Termino();
        else if (Detectar('-')) r -= Termino();
        else break;
    }
    return r;
}

double Termino() {
    double r = Unario();
    while (true) {
        if (Detectar('*')) {
            r *= Unario();
        } else if (Detectar('/')) {
            double d = Unario();
            if (d == 0) throw new DivideByZeroException("División por cero.");
            r /= d;
        } else break;
    }
    return r;
}

double Unario() {
    if (Detectar('-')) return -Unario();
    return Primario();
}

double Primario() {
    SaltarEspacios();
    if (Detectar('(')) {
        double v = Expresion();
        Consumir(')');
        return v;
    }
    int inicio = this.pos;
    while (this.pos < this.texto.Length &&
           (char.IsDigit(this.texto[this.pos]) || this.texto[this.pos] == '.'))
        this.pos++;
    if (this.pos == inicio)
        throw new FormatException($"Se esperaba un número en posición {this.pos}");
    return double.Parse(
        this.texto[inicio..this.pos],
        System.Globalization.CultureInfo.InvariantCulture);
}

```

```
}  
}
```

2.4 Prueba

csharp

```
var interp = new Interprete();  
  
Console.WriteLine(interp.Evaluar("3 + 4 * 2"));           // 11  
Console.WriteLine(interp.Evaluar("(3 + 4) * 2"));         // 14  
Console.WriteLine(interp.Evaluar("10 - 2 - 3"));          // 5 (asociatividad izquierda)  
Console.WriteLine(interp.Evaluar("--5"));                 // 5 (doble negación)  
Console.WriteLine(interp.Evaluar("2.5 * (1 + 1.5)"));      // 6.25
```

El intérprete funciona y es conciso. ¿Por qué entonces necesitaríamos algo más?

3. La limitación del intérprete y la utilidad del AST

El intérprete lee la expresión una sola vez y la descarta. Eso es suficiente para evaluar, pero hay cosas que no puede hacer:

- **Evaluar la misma expresión con distintas variables** (como $x * 2 + 1$) para muchos valores de x).
- **Mostrar la expresión** en distintos formatos: infijo con paréntesis, notación polaca, árbol visual.
- **Optimizar** antes de ejecutar: $0 * cualquierCosa$ siempre es cero.
- **Compilar** a otro lenguaje o a bytecode.

Para todo eso, necesitamos representar la expresión como una **estructura de datos** que podamos recorrer múltiples veces. Esa estructura es el **AST**.

4. Paso 2 — El compilador: RDP que construye un AST

4.1 Jerarquía de nodos

El AST es un árbol de objetos. Cada nodo representa una parte de la expresión. La jerarquía de clases refleja la estructura gramatical:

```
Nodo                (abstracto - todo nodo puede ser recorrido)
├─ Numero           (hoja - contiene un valor double)
├─ OperadorUnario   (abstracto - un nodo hijo llamado Operando)
│   └─ Negacion
└─ OperadorBinario  (abstracto - dos hijos: Izquierdo y Derecho)
    ├── Suma
    ├── Resta
    ├── Producto
    └─ Division
```

Las clases abstractas `OperadorUnario` y `OperadorBinario` concentran la estructura común. Las clases concretas sólo añaden su identidad — qué operación representan.

csharp

```
// — Nodo base —————

public abstract class Nodo {
    // Cada nodo concreto sabrá cómo recibir a un visitante (ver Paso 3)
}

// — Hoja —————

public class Numero(double valor) : Nodo {
    public double Valor => valor;

    public override string ToString() =>
        valor == Math.Truncate(valor)
            ? ((long)valor).ToString()
            : valor.ToString("G", System.Globalization.CultureInfo.InvariantCulture)
}

// — Operador con un solo operando —————

public abstract class OperadorUnario(Nodo operando) : Nodo {
    public Nodo Operando => operando;
}

public class Negacion(Nodo operando) : OperadorUnario(operando) {
    public override string ToString() => $"(-{this.Operando})";
}

// — Operador con dos operandos —————

public abstract class OperadorBinario(Nodo izquierdo, Nodo derecho) : Nodo {
    public Nodo Izquierdo => izquierdo;
    public Nodo Derecho => derecho;
}

public class Suma(Nodo izquierdo, Nodo derecho) : OperadorBinario(izquierdo, derecho) {
    public override string ToString() => $"({this.Izquierdo} + {this.Derecho})";
}

public class Resta(Nodo izquierdo, Nodo derecho) : OperadorBinario(izquierdo, derecho) {
    public override string ToString() => $"({this.Izquierdo} - {this.Derecho})";
}

public class Producto(Nodo izquierdo, Nodo derecho) : OperadorBinario(izquierdo, derecho) {
    public override string ToString() => $"({this.Izquierdo} * {this.Derecho})";
}
```

```
public class Division(Nodo izquierdo, Nodo derecho) : OperadorBinario(izquierdo, der
    public override string ToString() => $"({this.Izquierdo} / {this.Derecho})";
}
```

Los primary constructors hacen que `Suma(Nodo izq, Nodo der)` sea equivalente a declarar dos campos readonly y asignarlos en el constructor — en una sola línea. Los nodos son objetos de datos inmutables: una vez contruidos, no cambian.

4.2 El compilador

El compilador es el mismo RDP del intérprete, con un único cambio: donde antes retornaba `double`, ahora retorna `Nodo`. La lógica de análisis es idéntica.

csharp

```
public class Compilador {
    private string texto = "";
    private int pos = 0;

    public Nodo Compilar(string expresion) {
        this.texto = expresion;
        this.pos = 0;

        Nodo raiz = Expresion();

        SaltarEspacios();
        if (this.pos < this.texto.Length)
            throw new FormatException(
                $"Carácter inesperado '{this.texto[this.pos]}' en posición {this.pos}");

        return raiz;

        // — Utilidades (idénticas al intérprete) —————

        void SaltarEspacios() {
            while (this.pos < this.texto.Length && this.texto[this.pos] == ' ')
                this.pos++;
        }

        bool Detectar(char c) {
            SaltarEspacios();
            if (this.pos < this.texto.Length && this.texto[this.pos] == c) {
                this.pos++;
                return true;
            }
            return false;
        }

        char Consumir(char esperado) {
            SaltarEspacios();
            if (this.pos >= this.texto.Length)
                throw new FormatException($"Se esperaba '{esperado}' al final.");
            if (this.texto[this.pos] != esperado)
                throw new FormatException(
                    $"Se esperaba '{esperado}', se encontró " +
                    $"'{this.texto[this.pos]}' en posición {this.pos}.");
            return this.texto[this.pos++];
        }

        // — Gramática — retorna Nodo en lugar de double —————
    }
}
```

```

Nodo Expression() {
    Nodo izq = Termino();
    while (true) {
        if (Detectar('+')) izq = new Suma(izq, Termino());
        else if (Detectar('-')) izq = new Resta(izq, Termino());
        else break;
    }
    return izq;
}

Nodo Termino() {
    Nodo izq = Unario();
    while (true) {
        if (Detectar('*')) izq = new Producto(izq, Unario());
        else if (Detectar('/')) izq = new Division(izq, Unario());
        else break;
    }
    return izq;
}

Nodo Unario() {
    if (Detectar('-')) return new Negacion(Unario());
    return Primario();
}

Nodo Primario() {
    SaltarEspacios();
    if (Detectar('(')) {
        Nodo v = Expression();
        Consumir(')');
        return v;
    }
    int inicio = this.pos;
    while (this.pos < this.texto.Length &&
           (char.IsDigit(this.texto[this.pos]) || this.texto[this.pos] == '.'))
        this.pos++;
    if (this.pos == inicio)
        throw new FormatException($"Se esperaba un número en posición {this.pos}");
    return new Numero(double.Parse(
        this.texto[inicio..this.pos],
        System.Globalization.CultureInfo.InvariantCulture));
}
}

```

4.3 Ejecutar el AST con un método en la jerarquía

La primera forma de usar el árbol es agregarle un método `Evaluar` directamente a la jerarquía de nodos. Es simple y directa:

```
csharp

public abstract class Nodo {
    public abstract double Evaluar();
}

public class Numero(double valor) : Nodo {
    public double Valor => valor;
    public override double Evaluar() => valor;
}

public class Negacion(Nodo operando) : OperadorUnario(operando) {
    public override double Evaluar() => -this.Operando.Evaluar();
}

public class Suma(Nodo izquierdo, Nodo derecho) : OperadorBinario(izquierdo, derecho) {
    public override double Evaluar()
        => this.Izquierdo.Evaluar() + this.Derecho.Evaluar();
}

// ... ídem para Resta, Producto, Division
```

```
csharp

// Compilar una vez, evaluar múltiples veces
var compilador = new Compilador();
Nodo arbol = compilador.Compilar("(3 + 4) * (10 - 2)");

Console.WriteLine(arbol);           // ((3 + 4) * (10 - 2)) - ToString del árbol
Console.WriteLine(arbol.Evaluar()); // 56
```

Esto funciona. Pero ahora aparece el problema que el patrón Visitante viene a resolver.

5. Paso 3 — El patrón Visitante: operaciones sin modificar el árbol

5.1 El problema

Si queremos agregar una nueva operación — por ejemplo, un `Imprimir` que muestre el árbol indentado —, tenemos que modificar todas las clases: `Nodo`, `Numero`, `Negacion`, `Suma`, `Resta`, `Producto`, `Division`.

Con siete clases eso ya es tedioso. Si el AST tuviera veinte tipos de nodos (como en un lenguaje real), agregar cada nueva operación implicaría tocar veinte archivos.

El patrón **Visitante** invierte la dependencia: las operaciones se concentran en clases separadas (`Evaluador`, `Impresor`, etc.) y las clases del AST sólo necesitan un único método que las conecte con cualquier visitante presente y futuro.

5.2 El mecanismo: doble despacho

El visitante necesita saber el **tipo concreto** de cada nodo para llamar al método correcto. La solución es el **doble despacho**:

1. El código externo llama a `nodo.Aceptar(visitor)` — el tipo real del nodo determina qué `Aceptar` se ejecuta.
2. Dentro de `Aceptar`, el nodo llama a `visitante.VisitarXxx(this)` — ahora `this` tiene el tipo concreto conocido.

Así, con dos llamadas virtuales, el visitante recibe el tipo exacto sin ningún `if`/`switch`.

5.3 La interfaz del visitante

```
csharp

// T es el tipo del resultado que produce la operación:
//  IVisitante<double>  para evaluar
//  IVisitante<string>  para imprimir
//  IVisitante<Nodo>    para transformar el árbol

public interface IVisitante<T> {
    T VisitarNumero    (Numero    nodo);
    T VisitarNegacion  (Negacion   nodo);
    T VisitarSuma      (Suma       nodo);
    T VisitarResta     (Resta      nodo);
    T VisitarProducto  (Producto   nodo);
    T VisitarDivision  (Division   nodo);
}
```

5.4 Agregar `Aceptar` a la jerarquía

Cada clase concreta implementa `Aceptar` llamando al método de visita que le corresponde. Esto es todo lo que el árbol necesita saber sobre los visitantes:

```
csharp
```

```

public abstract class Nodo {
    public abstract T Aceptar<T>(IVisitante<T> visitante);
}

public class Numero(double valor) : Nodo {
    public double Valor => valor;
    public override T Aceptar<T>(IVisitante<T> v) => v.VisitarNumero(this);

    public override string ToString() =>
        valor == Math.Truncate(valor)
            ? ((long)valor).ToString()
            : valor.ToString("G", System.Globalization.CultureInfo.InvariantCulture)
}

public abstract class OperadorUnario(Nodo operando) : Nodo {
    public Nodo Operando => operando;
}

public abstract class OperadorBinario(Nodo izquierdo, Nodo derecho) : Nodo {
    public Nodo Izquierdo => izquierdo;
    public Nodo Derecho => derecho;
}

public class Negacion(Nodo operando) : OperadorUnario(operando) {
    public override T Aceptar<T>(IVisitante<T> v) => v.VisitarNegacion(this);
}

public class Suma(Nodo izquierdo, Nodo derecho) : OperadorBinario(izquierdo, derecho) {
    public override T Aceptar<T>(IVisitante<T> v) => v.VisitarSuma(this);
}

public class Resta(Nodo izquierdo, Nodo derecho) : OperadorBinario(izquierdo, derecho) {
    public override T Aceptar<T>(IVisitante<T> v) => v.VisitarResta(this);
}

public class Producto(Nodo izquierdo, Nodo derecho) : OperadorBinario(izquierdo, derecho) {
    public override T Aceptar<T>(IVisitante<T> v) => v.VisitarProducto(this);
}

public class Division(Nodo izquierdo, Nodo derecho) : OperadorBinario(izquierdo, derecho) {
    public override T Aceptar<T>(IVisitante<T> v) => v.VisitarDivision(this);
}

```

5.5 Los visitantes concretos

Evaluador — produce un **double**. La lógica que antes estaba repartida en los nodos ahora está

concentrada aquí:

csharp

```
public class Evaluador : IVisitante<double> {
    public double VisitarNumero (Numero n) => n.Valor;
    public double VisitarNegacion (Negacion n) => -n.Operando.Aceptar(this);
    public double VisitarSuma (Suma n) => n.Izquierdo.Aceptar(this) + n.Derecho.Aceptar(this);
    public double VisitarResta (Resta n) => n.Izquierdo.Aceptar(this) - n.Derecho.Aceptar(this);
    public double VisitarProducto (Producto n) => n.Izquierdo.Aceptar(this) * n.Derecho.Aceptar(this);

    public double VisitarDivision (Division n) {
        double divisor = n.Derecho.Aceptar(this);
        if (divisor == 0) throw new DivideByZeroException("División por cero en la expresión");
        return n.Izquierdo.Aceptar(this) / divisor;
    }
}
```

Impresor — produce un `string` con la notación infija completamente parentetizada. Hace explícita la estructura del árbol:

csharp

```
public class Impresor : IVisitante<string> {
    public string VisitarNumero (Numero n) => n.ToString();
    public string VisitarNegacion (Negacion n) => $"(-{n.Operando.Aceptar(this)})";
    public string VisitarSuma (Suma n) => Binario(n, "+");
    public string VisitarResta (Resta n) => Binario(n, "-");
    public string VisitarProducto (Producto n) => Binario(n, "*");
    public string VisitarDivision (Division n) => Binario(n, "/");

    private string Binario(OperadorBinario n, string op)
        => $"({n.Izquierdo.Aceptar(this)} {op} {n.Derecho.Aceptar(this)})";
}
```

ImpresionArbol — produce un `string` que muestra la estructura del árbol con indentación, útil para depuración:

csharp

```

public class ImpresionArbol : IVisitante<string> {
    public string VisitarNumero (Numero n) => n.ToString();

    public string VisitarNegacion (Negacion n) =>
        $"neg\n{Indentar(n.Operando.Aceptar(this))}";

    public string VisitarSuma (Suma n) => Rama(n, "+");
    public string VisitarResta (Resta n) => Rama(n, "-");
    public string VisitarProducto (Producto n) => Rama(n, "*");
    public string VisitarDivision (Division n) => Rama(n, "/");

    private string Rama(OperadorBinario n, string op) {
        string izq = n.Izquierdo.Aceptar(this);
        string der = n.Derecho.Aceptar(this);
        return $"{op}\n{Indentar(izq, "| ")}\n{Indentar(der, " ")}";
    }

    private static string Indentar(string texto, string prefijo = " ") {
        var lineas = texto.Split('\n');
        return lineas[0] + (lineas.Length > 1
            ? "\n" + string.Join("\n", lineas[1..].Select(l => prefijo + l))
            : "");
    }
}

```

5.6 Uso integrado

csharp

```

var compilador    = new Compilador();
var evaluador     = new Evaluador();
var impresor      = new Impresor();
var arbolVisor    = new ImpresionArbol();

string[] expresiones = [
    "3 + 4 * 2",
    "(3 + 4) * 2",
    "10 - 2 - 3",
    "--5",
    "2.5 * (1 + 1.5)",
    "10 / (2 + 3)",
];

foreach (string expr in expresiones) {
    Nodo arbol = compilador.Compilar(expr);

    Console.WriteLine($"Entrada:    {expr}");
    Console.WriteLine($"Infijo:      {arbol.Aceptar(impresor)}");
    Console.WriteLine($"Resultado:  {arbol.Aceptar(evaluador)}");
    Console.WriteLine($"Árbol:");
    Console.WriteLine(arbol.Aceptar(arbolVisor));
    Console.WriteLine();
}

```

Salida para $(3 + 4) * 2$:

```

Entrada:    (3 + 4) * 2
Infijo:      ((3 + 4) * 2)
Resultado:  14
Árbol:
*
├─ +
│   └─ 3
│       └─ 4
└─ 2

```

Agregar un nuevo visitante —por ejemplo, uno que cuente la cantidad de operaciones, u otro que genere código ensamblador— sólo requiere crear una nueva clase que implemente

`IVisitante<T>`. Las clases del AST no se tocan.

6. Qué ilustra cada mecanismo

Mecanismo	Dónde aparece	Efecto
Clase <code>abstract</code>	<code>Nodo</code> , <code>OperadorUnario</code> , <code>OperadorBinario</code>	No se pueden instanciar directamente; definen estructura común
Primary constructor	<code>Numero(double valor)</code> , <code>Suma(Nodo izq, Nodo der)</code> ,...	Declara campos y constructor en una sola línea
Herencia <code>protected</code>	<code>Operando</code> , <code>Izquierdo</code> , <code>Derecho</code> en las abstractas	Los hijos son accesibles en subclases y visitantes, no en código externo
<code>override</code>	Cada clase concreta implementa <code>Aceptar</code>	Despacho dinámico: <code>nodo.Aceptar(v)</code> ejecuta el <code>Aceptar</code> del tipo real
Funciones locales	<code>SaltarEspacios</code> , <code>Detectar</code> , <code>Consumir</code> , <code>Expresion</code> ,...	Encapsulan lógica privada del método; capturan <code>this.pos</code> y <code>this.texto</code>
Recursión de funciones locales	<code>Expresion</code> llama a <code>Termino</code> , que llama a <code>Unario</code> , que puede llamar a <code>Expresion</code>	Implementa la gramática directamente; la pila de llamadas es el árbol
Interfaz genérica <code>IVisitante<T></code>	<code>Evaluador :</code> <code>IVisitante<double></code> , <code>Impresor :</code> <code>IVisitante<string></code>	Una sola interfaz para operaciones que producen distintos tipos
Doble despacho	<code>nodo.Aceptar(v)</code> → <code>v.VisitarXxx(this)</code>	El tipo del nodo y el tipo del visitante determinan juntos el método ejecutado